

AI FOOD ANALYZER

Müəllim qarşısında proqramın canlı nümayişi və müdafiə ssenarisi

Yoxlanmış nəticə	Dəyər
Automated tests	45 passed
Application coverage	89%
Sequential benchmark	2.56 s
Concurrent benchmark	0.50 s
Measured speedup	5.11x
Docker API startup	Successful

Məqsəd: Bu sənədi müdafiə zamanı ardıcılıq kimi istifadə et. Əvvəl təqdimatı göstər, sonra aşağıdakı addımlarla proqramı canlı nümayiş etdir.

1. Müdafiədən əvvəl hazırlıq

Canlı demo zamanı vaxt itirməmək üçün aşağıdakıları əvvəlcədən hazır saxla.

- PyCharm-da topic-2-food-analyzer layihəsini aç.
- Terminalin layihənin əsas qovluğunda olduğuna əmin ol.
- PowerPoint təqdimatını əvvəlcədən aç.
- Docker Desktop istifadə edəcəksənsə, onu əvvəlcədən işə sal.
- Real API key yoxdursa, əsas meal demo üçün offline mode istifadə et.
- Brauzerdə FastAPI üçün yeni tab hazır saxlaya bilərsən: <http://localhost:8000/docs>.

Müdafiədə deyəcəyin söz:

Salam. Mənim layihəm AI Food Analyzer-dir. Proqram yemək şəkildən inqrediyentləri müəyyən etmək, porsiya miqdarını qiymətləndirmək və kalori ilə makronutrientləri hesablamaq üçün hazırlanıb. Layihədə AI hissəsi ilə yanaşı CLI, HTTP API, asynchronous processing, concurrency, cache, PostgreSQL, automated testing və Docker kimi Software Engineering komponentləri də istifadə olunub.

Təvsiyə olunan demo ardıcılığı

- 1) Offline meal analysis
- 2) Non-meal image handling
- 3) Automated tests
- 4) Test coverage
- 5) Concurrency benchmark
- 6) FastAPI / Swagger
- 7) Docker - vaxt qalarsa

2. Addım-addım canlı demo

ADDIM 1 - Virtual environment

Terminal əmri

```
.venv\Scripts\activate
```

Qısa izah: Python dependencies layihənin lokal virtual environment-indən istifadə edəcək. Əgər terminalda artıq (.venv) görünürsə, bu əmri yenidən yazmağa ehtiyac yoxdur.

Müdafiədə deyəcəyin söz:

İlk olaraq layihənin virtual environment-ini aktiv edirəm. Bu, layihənin tələb etdiyi Python paketlərini izolə olunmuş mühitdə istifadə etməyə imkan verir.

ADDIM 2 - Yemək şəklinin offline analizi

Terminal əmri

```
python demo_ai.py --offline --image data/rice_chicken.png
```

Qısa izah: Bu deterministic offline demo-dur. Xarici VLM və USDA API key tələb etmir. Sample mapping əsasında ingredient və nutrition workflow-nu nümayiş etdirir.

Ekranda nə görəcəksən: Rice və grilled chicken ingredientləri, onların qram miqdarı və ümumi nutrition nəticəsi. Yoxlanmış nümunədə total təxminən 482 kcal, 51.4 g protein, 50.4 g carbs və 5.9 g fat göstərilir.

Müdafiədə deyəcəyin söz:

İndi nümunə yemək şəklini analiz edirəm. Offline mode istifadə etdiyim üçün demo xarici API və internet bağlantısından asılı deyil. Bu mode real VLM inference deyil; layihənin workflow-nu deterministik şəkildə nümayiş etdirir.

ADDIM 3 - Yemək olmayan şəkli yoxla

Terminal əmri

```
python demo_ai.py --offline --image data/no_meal_blue.png
```

Qısa izah: Bu test proqramın yalnız düzgün yemək şəkillərində deyil, tanınmayan/non-meal input-da da idarə olunan davranış göstərdiyini nümayiş etdirir.

Ekranda nə görəcəksən: Meal not recognized in image.

Müdafiədə deyəcəyin söz:

İndi sistemin yemək olmayan şəkli necə idarə etdiyini göstərirəm. Proqram exception ilə dayanmaq əvəzinə istifadəçiyə yeməyin tanınmadığını bildirir.

ADDIM 4 - Automated testləri işə sal

Terminal əmri

```
python -m pytest -v
```

Qısa izah: Pytest ilə API, cache, CLI, history service, nutrition service və əsas service daxil olmaqla layihənin əsas hissələri yoxlanılır.

Ekranda nə görəcəksən: 45 passed. Starlette/AnyIO ilə bağlı deprecation warning görünə bilər; bu test failure deyil.

Müdafiədə deyəcəyin söz:

Layihə üçün automated test suite hazırlamışam. İndi bütün testləri birlikdə işə salıram. Bu, komponentlərin gözlənilən davranışı verdiyini yoxlayır.

ADDIM 5 - Test coverage göstər

Terminal əmri

```
python -m pytest --cov=app --cov-report=term-missing
```

Qısa izah: Coverage testlərin app qatındakı kodun nə qədərini icra etdiyini ölçür. Layihə tələbi minimum 60%-dir.

Ekranda nə görəcəksən: TOTAL coverage: 89%.

Müdafiədə deyəcəyin söz:

İndi test coverage-i göstərirəm. Layihədə tələb olunan minimum göstərici 60 faizdir, mənim app qatında ölçülmüş coverage nəticəm 89 faizdir.

ADDIM 6 - Concurrency benchmark

Terminal əmri

```
python benchmark.py
```

Qısa izah: Benchmark 5 simulated I/O nutrition lookup-u sequential və concurrent formada müqayisə edir. Concurrent hissədə asyncio.gather() yanaşmasının faydası göstərilir.

Ekranda nə görəcəksən: Sequential: 2.56 s; Concurrent: 0.50 s; Speedup: 5.11x.

Müdafiədə deyəcəyin söz:

Nutrition lookup-lar bir-birindən asılı olmayan I/O əməliyyatlarıdır. Buna görə onları sequential yerinə asyncio.gather ilə concurrent icra edirəm. Benchmark deterministik olması üçün simulated I/O delay istifadə edir.

3. FastAPI nümayişi

ADDIM 7 - HTTP serveri işə sal

Terminal əmri

```
uvicorn app.api:app --host 0.0.0.0 --port 8000
```

Qısa izah: Uvicorn FastAPI tətbiqini 8000 portunda işə salır.

Ekranı nə görə bilərsiniz: Uvicorn running on http://0.0.0.0:8000

Müdafiədə deyəcəyiniz söz:

Layihə yalnız CLI deyil, HTTP API də təqdim edir. İndi FastAPI serverini işə salıram.

Sonra brauzerdə aç:

```
http://localhost:8000/docs
```

Müdafiədə deyəcəyiniz söz:

Bu FastAPI tərəfindən avtomatik yaradılan Swagger interface-dir. Burada API endpoint-lərini görə və test edə bilərsiniz.

GET /health

Swagger-də GET /health seç → Try it out → Execute.

```
{"status": "ok"}
```

Müdafiədə deyəcəyiniz söz:

Health endpoint serverin işlək olduğunu yoxlamaq üçün istifadə olunur. Status ok cavabı API-nin işlədiyini göstərir.

POST /analyze

Bu endpoint JPEG və PNG şəkillərini qəbul edir. Fayl tipi və maksimum ölçü validation-dan keçir. Default maksimum ölçü 5 MB-dir.

Müdafiədə deyəcəyiniz söz:

Analyze endpoint meal image upload qəbul edir. Sistem yalnız JPEG və PNG fayllarına icazə verir və default olaraq 5 MB maksimum ölçünü yoxlayır. Dəstəklənməyən fayl üçün HTTP 400, həddindən böyük fayl üçün HTTP 413 qaytarılır.

Qeyd: Real /analyze AI workflow-u provider API credentials tələb edə bilər. API key hazır deyilsə, müdafiədə POST /analyze üzərindən real VLM nəticəsi almağa çalışın; endpoint-i və validation məntiqini göstərmək kifayətdir.

4. Docker nümayişi - vaxt qalarsa

ADDIM 8 - Docker container

Terminal əmri

```
docker run --name food-analyzer-demo -p 8000:8000 food-analyzer
```

Qısa izah: Əvvəldən build edilmiş food-analyzer image-dən container başladılır və host-un 8000 portu container-in 8000 portuna bağlanır.

Ekranda nə görəcəksən: FastAPI/Uvicorn container daxilində başlayır.

Müdafiədə deyəcəyin söz:

Sonda tətbiqin containerized formada da işlədiyini göstərirəm. Docker layihənin eyni runtime mühitində başqa sistemlərdə də daha rahat işlədilməsinə kömək edir.

Əgər eyni adlı container əvvəl mövcuddursa, demo üçün başqa ad seç və ya əvvəlki stopped container-i sil.

5. Layihənin iş prinsipini qısa izah et

Müdafiədə deyəcəyiniz söz:

İstifadəçi meal image verir. VLM görünən inqrediyentləri və onların təxmini qram miqdarını müəyyən edir. Sonra nutrition service hər ingredient üçün nutrition məlumatını əldə edir. Bu lookup-lar müstəqil olduğuna görə asyncio.gather ilə concurrent icra olunur. Nutrition calculator porsiya miqdarına uyğun olaraq kalori, protein, carbohydrates və fat dəyərlərini hesablayır. Nəticələr HTTP API və ya CLI vasitəsilə istifadəçiyə təqdim oluna və analysis history PostgreSQL-də saxlanıla bilər.

Əsas komponentlər

Komponent	Funksiya
ai/vlm.py	Şəkildən ingredient və estimated grams əldə edən AI qatı
app/service.py	Ümumi analysis workflow-nu koordinasiya edir
app/nutrition_service.py	Nutrition lookup və concurrency
app/cache.py	Təkrar nutrition lookup-ları azaltmaq üçün cache
app/history_service.py	Analysis history-nin saxlanması
app/cli.py	Command-line interface
app/api.py	FastAPI HTTP interface

6. Müəllimin verə biləcəyi suallar və qısa cavablar

Sual: Niyə asyncio.gather istifadə etmisən?

Cavab: Çünki hər ingredient üçün nutrition lookup müstəqil I/O əməliyyatıdır. asyncio.gather onları concurrent icra etməyə və ümumi gözləmə vaxtını azaltmağa imkan verir.

Sual: 5.11x speedup həmişə olacaq?

Cavab: Xeyr. 5.11x nəticəsi simulated I/O ilə aparılmış controlled benchmark-a aiddir. Real API performansını network latency, provider response time və digər amillərdən asılıdır.

Sual: Cache nə üçün lazımdır?

Cavab: Eyni ingredient üçün nutrition məlumatı təkrar lazım olduqda external lookup sayını azaltmaq və response time-i yaxşılaşdırmaq üçün. Default TTL 24 saatdır.

Sual: PostgreSQL nə üçün istifadə olunur?

Cavab: Analysis history-ni persistent saxlamaq üçün: timestamp, image path, ingredients və nutrition totals kimi məlumatlar saxlanıla bilər.

Sual: Offline demo real AI-dır?

Cavab: Xeyr. Offline demo deterministic sample mapping-dir və xarici API olmadan workflow-u nümayiş etdirmək üçündür. Real VLM workflow provider API key tələb edir.

Sual: Niyə həm CLI, həm API var?

Cavab: CLI terminaldan sürətli istifadə və test üçün, HTTP API isə tətbiqin başqa client və sistemlərlə integrasiyası üçün nəzərdə tutulub.

Sual: Test nəticən nədir?

Cavab: Full test suite-də 45 test keçir və app qatında ölçülmüş coverage 89%-dir.

Sual: Non-meal image gələndə nə olur?

Cavab: Sistem uncontrolled failure vermir; ingredient siyahısı boş qalır və istifadəçiyə meal-in tanınmadığı bildirilir.

Sual: Docker nə verir?

Cavab: Tətbiqi dependencies və runtime ilə birlikdə container-də paketləyir, bununla müxtəlif mühitlərdə daha reproducible şəkildə işə salmaq mümkün olur.

7. 60-90 saniyəlik yekun müdafiə sözü

Müdafiədə deyəcəyiniz söz:

AI Food Analyzer layihəsində məqsədim təqdim edilmiş AI komponentlərini tam Software Engineering tətbiqinə çevirmək idi. Sistem meal image-dən ingredient və portion estimate alır, nutrition məlumatını emal edir və calories, protein, carbohydrates və fat totals hesablayır. Nutrition lookup-ların performansını yaxşılaşdırmaq üçün asynchronous programming və asyncio.gather istifadə olunur, təkrar sorğuları azaltmaq üçün cache mövcuddur və analysis history PostgreSQL-də saxlanıla bilər. Layihə həm CLI, həm də FastAPI HTTP API təqdim edir. Validation, error handling, automated testing və Docker containerization da implementasiya olunub. Full test suite-də 45 test keçir, app coverage 89 faizdir. Controlled concurrency benchmark-da sequential icra 2.56 saniyə, concurrent icra 0.50 saniyə olub və 5.11 dəfə speedup ölçülüb. Beləliklə, layihə yalnız AI nəticəsi yaratmır, həm də test edilə bilən, modular və containerized Software Engineering arxitekturasını nümayiş etdirir.

8. Müdafiə zamanı yadda saxla

- 5.11x nəticəsini real API üçün universal nəticə kimi təqdim etmə; controlled benchmark olduğunu de.
- Offline demo-nun real VLM inference olmadığını açıq şəkildə de.
- API key yoxdursa, real VLM command-ını canlı demo-nun əsas hissəsinə çevirmə.
- Warning görünsə, onu test failure kimi təqdim etmə. Əsas nəticə: 45 passed.
- Əvvəl işləyən, təhlükəsiz demo-ları göstər; Docker-i sona saxla.
- Kod haqqında sual veriləndə əvvəl workflow-u izah et, sonra konkret faylı göstər.

Qısa demo planı

Vaxt	Nə göstərilir
0-1 dəq.	Layihənin məqsədi və architecture
1-2 dəq.	Offline meal analysis
2-3 dəq.	Non-meal handling
3-4 dəq.	45 automated tests
4-5 dəq.	89% coverage
5-6 dəq.	Concurrency benchmark
6-8 dəq.	FastAPI / Swagger
Əlavə vaxt	Docker